Section 3
Exercises

Example #1:-

Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.

Model this tollbooth with a class called `tollBooth`. The two data items are a type `unsigned int` to hold the total number of cars, and a type `double` to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called `payingCar()` increments the car total and adds 0.50 to the cash total. Another function, called `nopayCar()`, increments the car total but adds nothing to the cash total. Finally, a member function called `display()` displays the two totals. Make appropriate member functions `const`.

Include a program to test this class. This program should allow the user to push one key to count a paying car, and another to count a nonpaying car. Pushing the Esc key should cause the program to print out the total cars and total cash and then exit.

```
#include <iostream>
using namespace std;
#include <conio.h>
const char ESC = 27; //escape key ASCII code
const double TOLL = 0.5; //toll is 50 cents
```

```
class tollBooth
{
private:
    unsigned int totalCars; //total cars passed today
    double totalCash; //total money collected today
public: //constructor
    tollBooth() : totalCars(0), totalCash(0.0)
    { }
    void payingCar(){  //a car paid
        totalCars++;
        totalCash += TOLL;
    }
    void nopayCar(){  //a car didn't pay
        totalCars++;
    }
    void display() const {  //display totals
        cout << "\nCars=" << totalCars
                            << ", cash=" << totalCash << endl;
    }
};
```

```
int main()
{
    tollBooth booth1; //create a toll booth
    char ch;
    cout << "\nPress 0 for each non-paying car,"
         << "\n 1 for each paying car,"
             << "\n Esc to exit the program.\n";
do
{
    ch = getche(); //get character
        if( ch == '0' ) //if it's 0, car didn't pay
            booth1.nopayCar();
        if( ch == '1' ) //if it's 1, car paid
            booth1.payingCar();
}
    while( ch != ESC );    //exit loop on Esc key
    booth1.display(); //display totals
    return 0;
}
```

Output

Example #2:-

Create a class called `time` that has separate `int` member data for hours, minutes, and seconds. One constructor should initialize this data to 0, and another should initialize it to fixed values. Another member function should display it, in 11:59:59 format. The final member function should add two objects of type `time` passed as arguments.

A `main()` program should create two initialized `time` objects (should they be `const`?) and one that isn't initialized. Then it should add the two initialized values together, leaving the result in the third `time` variable. Finally it should display the value of this third variable. Make appropriate member functions `const`.

```
class time
{
    private:
        int hrs, mins, secs;
    public:
        time() : hrs(0), mins(0), secs(0){ } //no-arg constructor
        time(int h, int m, int s) : hrs(h), mins(m), secs(s){ } //3-arg constructor

    void display() const {  //format 11:59:59
        cout << hrs << ":" << mins << ":" << secs;
    }
    void add_time(time t1, time t2){ //add two times
        secs = t1.secs + t2.secs; //add seconds
        if( secs > 59 ){   //if overflow,
            secs -= 60;    // carry a minute
            mins++;
        }
        mins += t1.mins + t2.mins; //add minutes
        if( mins > 59 ) {  //if overflow,
            mins -= 60;    // carry an hour
            hrs++;
        }
        hrs += t1.hrs + t2.hrs; //add hours
    }
};
```

```
int main()
{
    const time time1(5, 59, 59); //creates and initialze
    const time time2(4, 30, 30); // two times
    time time3; //create another time
    time3.add_time(time1, time2); //add two times
    cout << "time3 = ";
    time3.display(); //display result
    cout << endl;
    return 0;
}
```

Output

Constant Objects

- An object of a class may be declared to be **const**, just like any other C++ variable. For example:

```
const Date birthday(7, 3, 1969);
```

- Only constant member functions can be called for a constant object:

```
birthday.set_month(12);      // Syntax error.
```